

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

GitHub Repository: https://github.com/Hasini923/Deep_Learning/tree/main/RNN_LSTM_GRU

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Preprocessing Output

Input tensor shape:

Training : (2470, 128, 9)

Validation : (530, 128, 9)

Testing : (700, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

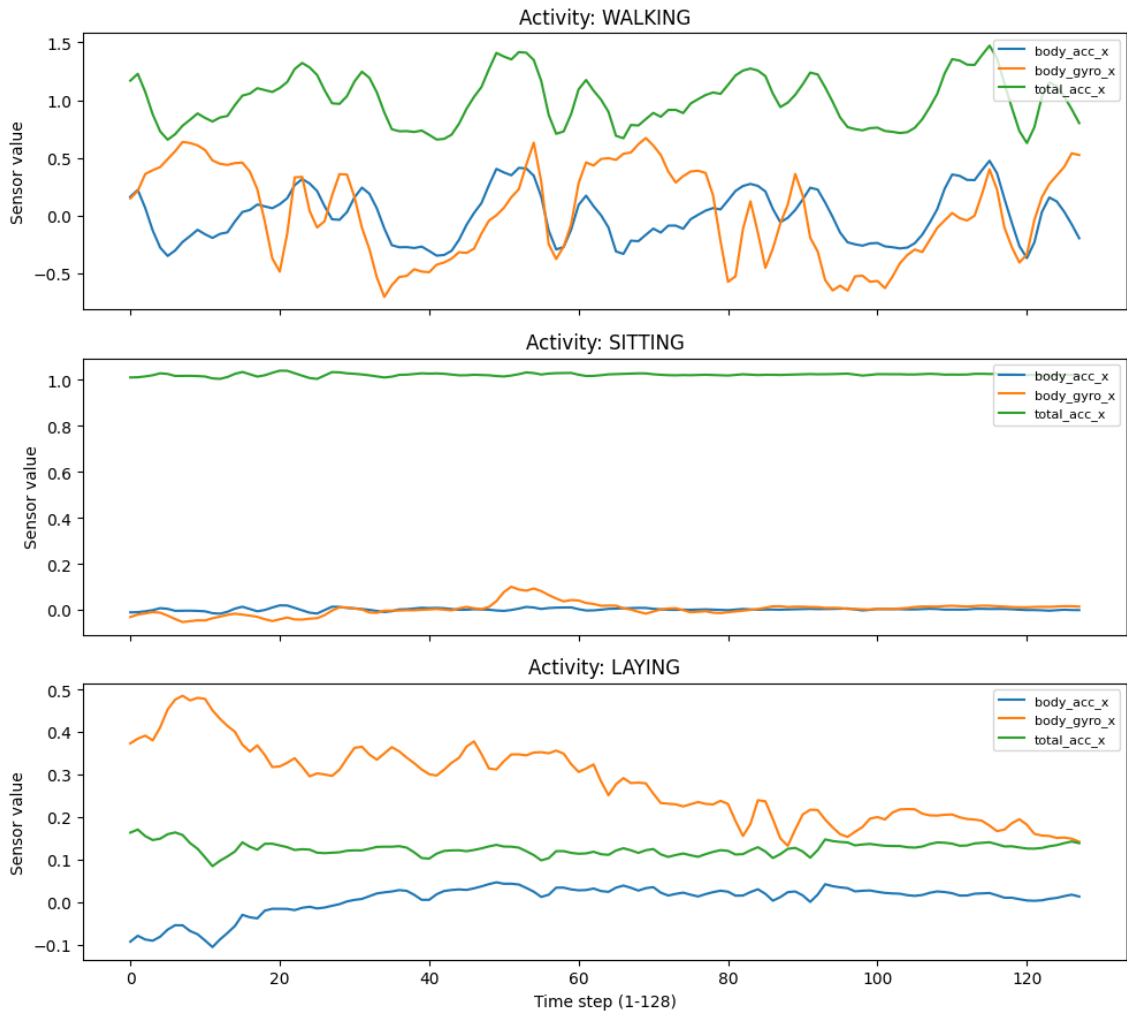
Class distribution (train): 0:408 1:338 2:348 3:438 4:456 5:482
Class distribution (val) : 0:87 1:73 2:75 3:94 4:98 5:103
Class distribution (test) : 0:113 1:111 2:105 3:121 4:129 5:121

Inference: A 3000-window subset of the raw inertial signals was drawn, giving 2470 training, 530 validation and 700 test windows, each of shape 128×9 , across all six activity classes. The per-class counts in each split are close to balanced (roughly 15–18% of the split per class), so no class dominates the loss during training and evaluation metrics are not skewed by class imbalance.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time



Inference:

1. WALKING shows a strong, repeating oscillatory pattern in `body_acc_x` and `body_gyro_x` with amplitude swings of roughly ± 0.5 , consistent with the periodic limb motion of walking. SITTING and LAYING are almost flat, with `total_acc_x` sitting at a near-constant level (around 1.0 for SITTING, around 0.15 for LAYING) and negligible gyroscope activity, since the body is stationary.
2. WALKING_UPSTAIRS and WALKING_DOWNSTAIRS (not shown here but present in the full signal set) exhibit the same kind of periodic oscillation as WALKING, since all three are locomotion activities; SITTING and STANDING similarly resemble each other as static postures. This is exactly the confusion pattern later seen in the confusion matrices.

3. The ordering of the 128 measurements encodes the temporal shape of the motion cycle (acceleration rising then falling repeatedly for walking, or staying flat for static postures). Shuffling the time steps would destroy this cyclic structure and make it impossible for a recurrent model to tell a walking-type activity from a static one using shape alone.

7. Vanilla RNN Architecture

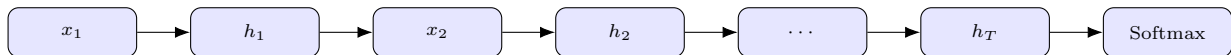
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

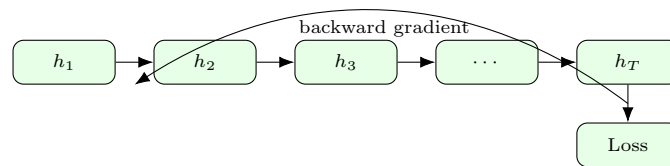
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Output (manual vs. programmatic):

Manually computed hidden states:

$h_1 = 0.336376$

$h_2 = 0.616352$

$h_3 = 0.599958$

Programmatically computed hidden states:

$h_1 = 0.336376$

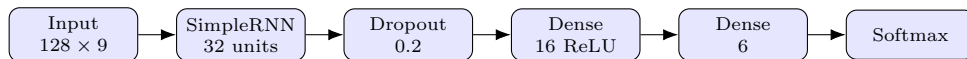
$h_2 = 0.616352$

$h_3 = 0.599958$

Inference: The manually computed values match the values produced by a `SimpleRNNCell` with the same weights exactly (to six decimal places), confirming that the recurrence $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ is implemented correctly. Note that $h_3 < h_2$ despite $x_3 = 0.2$ being smaller than $x_2 = 0.7$: because h_{t-1} feeds back into every step, the hidden state does not track the input alone but a nonlinear mixture of the current input and all previous history.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

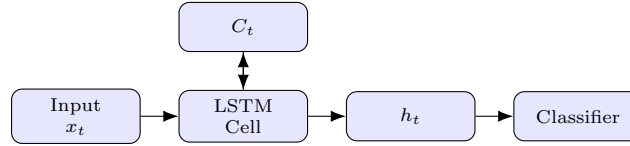
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

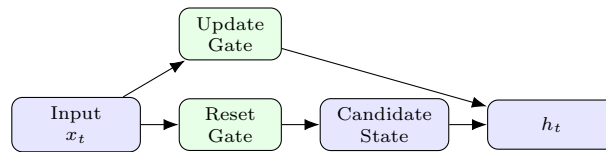
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes	Trainable Parameters
RNN	SimpleRNN	32	6	1,974
LSTM	LSTM	32	6	6,006
GRU	GRU	32	6	4,758

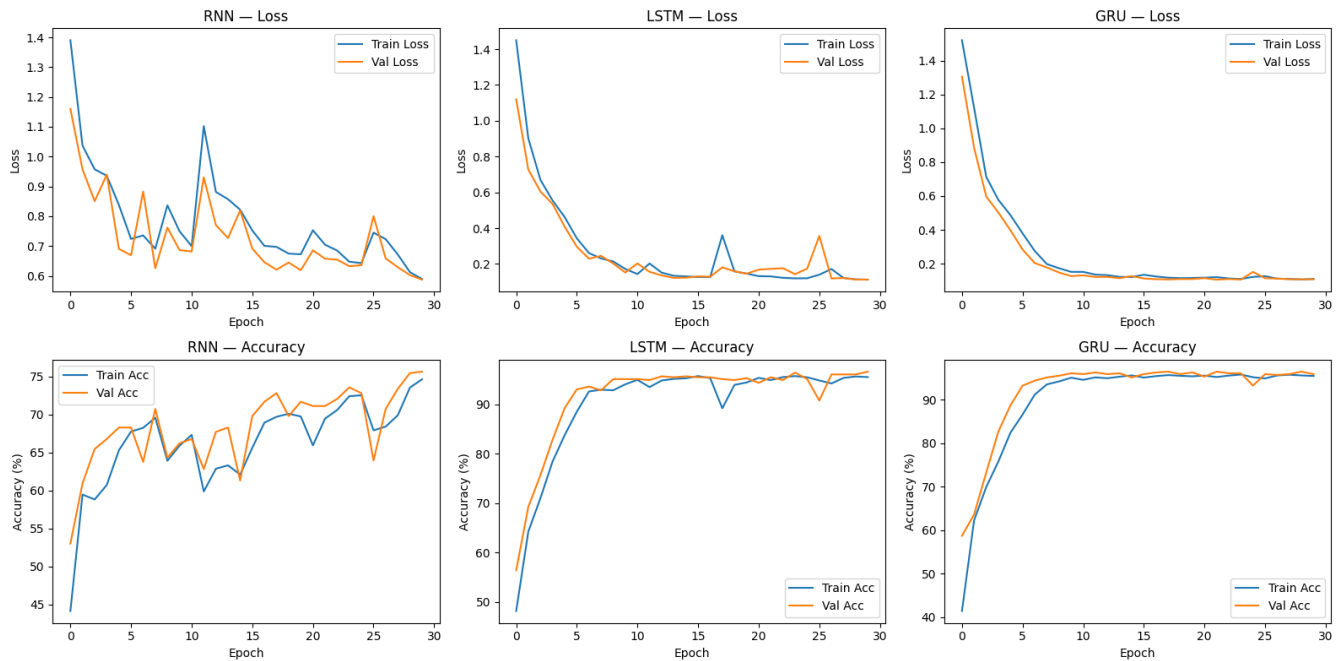
All other experimental conditions (input shape, dropout, dense head, optimizer, batch size, epochs, loss) were kept identical across the three models. GRU has fewer parameters than LSTM at the same unit count because it uses two gates (reset, update) instead of LSTM's three gates plus a separate cell state.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss



Inference:

- **RNN convergence:** The RNN's loss and accuracy curves are visibly noisy across all 30 epochs, with repeated spikes (e.g. around epochs 11 and 25) and a final training/validation accuracy of only $\approx 75\%$. This reflects the vanishing-gradient limitation of a plain recurrent cell over a 128-step sequence – it struggles to converge smoothly.
- **LSTM convergence:** Loss drops sharply within the first 5 epochs and validation accuracy reaches $\approx 95\%$ by epoch 10, after which it plateaus with only minor fluctuations (a small spike near epoch 17 and 25 that quickly recovers). Training and validation curves track each other closely throughout.
- **GRU convergence:** GRU converges even faster and more smoothly than LSTM, reaching $\approx 90\%$ validation accuracy by epoch 5 and $\approx 95\%$ by epoch 10, with almost no oscillation for the remainder of training.
- **Overfitting/underfitting:** For both LSTM and GRU, training and validation curves stay close together throughout, with validation accuracy never persistently below training accuracy – there is no evidence of overfitting. The RNN's noisy, non-monotonic curves and lower final accuracy are better described as underfitting/optimization difficulty rather than overfitting.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

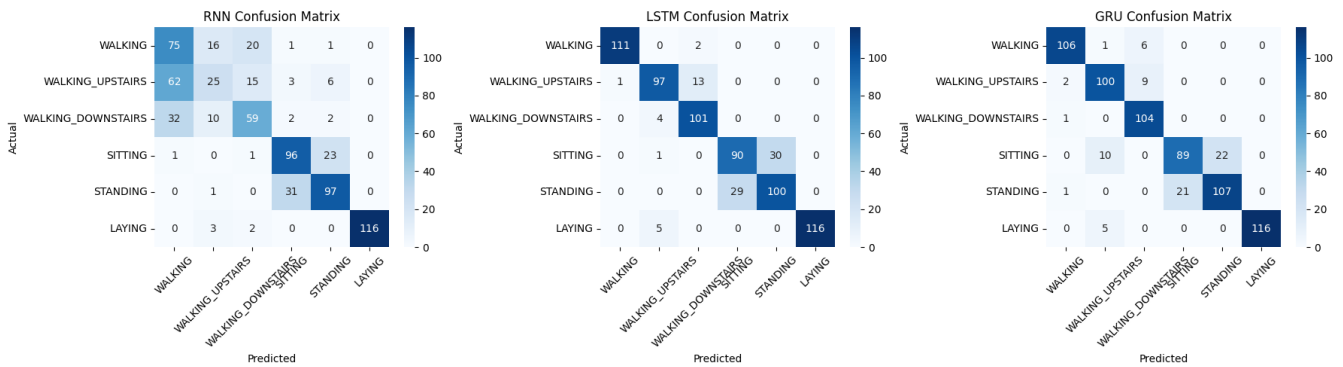
For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	66.86	87.86	88.86
Macro Precision (%)	66.30	88.23	88.97
Macro Recall (%)	65.91	88.26	89.22
Macro F1 (%)	65.04	88.20	88.99
Parameters	1,974	6,006	4,758
Training Time (s)	32.38	32.50	27.59

Inference: On the held-out test set, both gated architectures outperform the plain RNN by more than 20 percentage points on every metric. GRU edges out LSTM on accuracy and F1 (88.86% vs. 87.86%, 88.99% vs. 88.20%) while using about 20% fewer parameters and training in the least wall-clock time of the three (27.59 s), making it the best accuracy-per-cost model in this run.

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix



Inference:

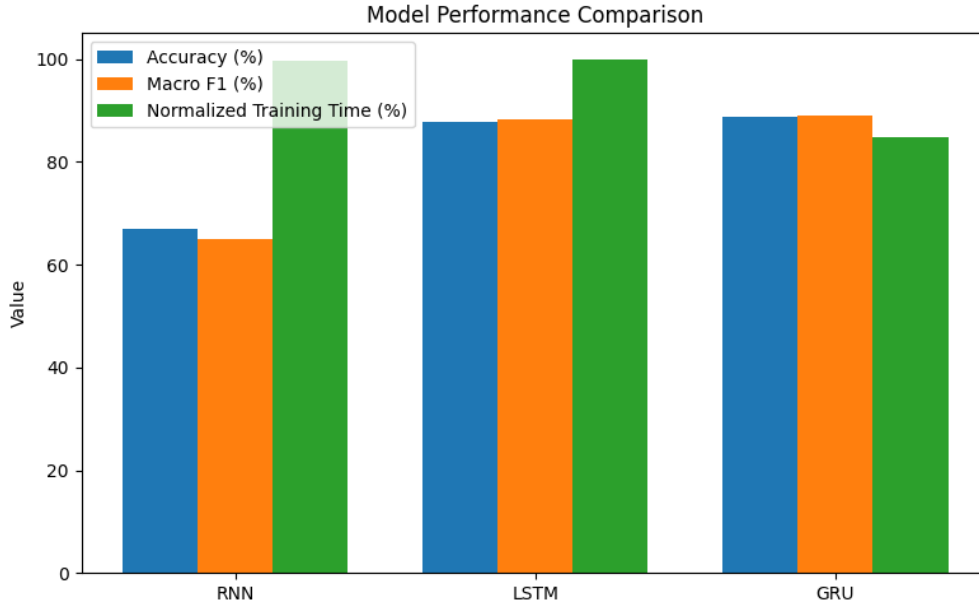
- **Highest recognition rate:** LAYING is recognised perfectly by all three models (116/116 correct), since it is the most physically distinct (near-static, low-gravity-axis) activity.
- **Largest misclassification:** For the RNN, WALKING_UPSTAIRS is the weakest class – only 25 of 111 samples are correctly labelled, with 62 confused as WALKING and 15 as WALKING_DOWNSTAIRS. For LSTM and GRU the worst class is SITTING, with 30 and 22 samples respectively misclassified as STANDING.
- **Frequently confused pairs:** WALKING ↔ WALKING_UPSTAIRS ↔ WALKING_DOWNSTAIRS form one confusable cluster (all locomotion), and SITTING ↔ STANDING form a second cluster (both static postures with similar sensor signatures).
- **Consistency across models:** The same two confusable clusters appear in all three confusion matrices, but their severity drops sharply from RNN to LSTM/GRU – e.g. the three walking classes are almost cleanly separated by LSTM and GRU (off-diagonal counts in single digits), while the RNN confuses them heavily. This shows the errors are systematic (driven by genuine sensor-signal similarity between activities) rather than random, and that gated units are substantially better at resolving this ambiguity.

16. RNN vs. LSTM vs. GRU Comparison

The completed comparison is as follows.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	32.38	32.50	27.59
Test F1-score (%)	65.04	88.20	88.99

Plot 5: Model Performance Comparison



Inference: The RNN is the cheapest model (fewest parameters) but also the worst performer, so it offers poor value even at low cost. LSTM and GRU sit close together on accuracy and F1, but GRU reaches that performance with $\sim 21\%$ fewer parameters than LSTM and the shortest training time of the three – it dominates LSTM on this task (better performance *and* lower cost). This illustrates that raising model complexity only pays off up to the point where the extra gating actually resolves ambiguity the simpler model cannot (RNN $\rightarrow$ GRU/LSTM); beyond that, the more complex gate set of LSTM does not buy extra performance over GRU on this dataset.

17. Effect of Sequence Length

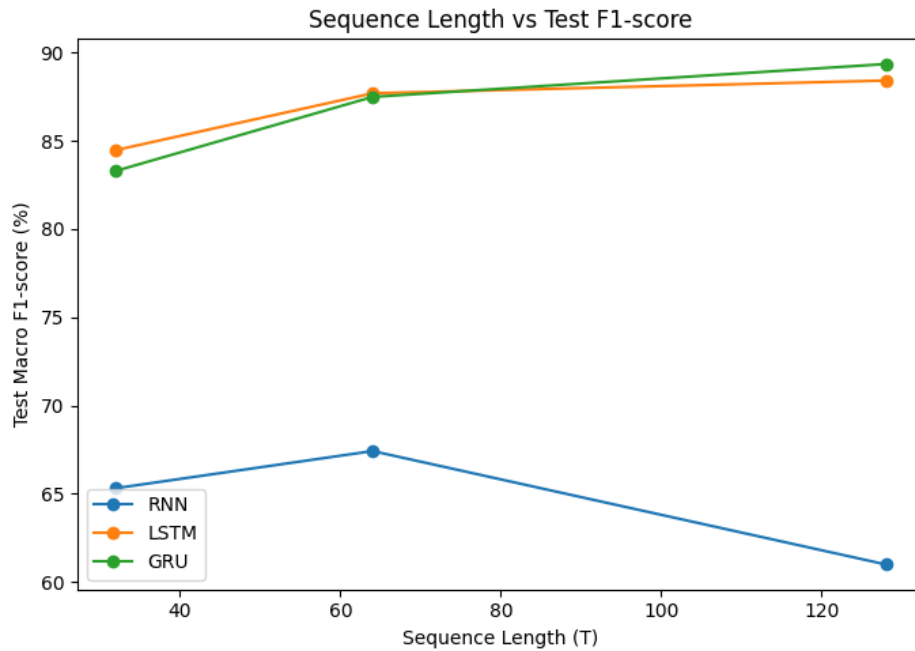
To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	65.31	84.47	83.29
64	67.42	87.69	87.48
128	60.99	88.41	89.34

Plot 6: Sequence Length vs. Test F1-score



Inference: LSTM and GRU both improve as more temporal context is given (F1 rises from ~84–83% at $T = 32$ to ~88–89% at $T = 128$), showing they can exploit the additional history to disambiguate similar activities. The plain RNN behaves differently: its F1 improves slightly from $T = 32$ to $T = 64$ (65.31% $\rightarrow$ 67.42%) but then *drops* at $T = 128$ (60.99%), which is consistent with the vanishing-gradient problem – beyond a certain sequence length the RNN can no longer propagate useful gradient information back through all time steps, so extra context stops helping and starts hurting optimization.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Dataset Used

A pre-curated 5-class subset of UCF101 (594 train + 224 test clips) was used and three classes were sampled from it for a lab-scale run:

Classes available: ['CricketShot', 'PlayingCello', 'Punch',
'ShavingBeard', 'TennisSwing']

Total videos: 818 (594 train, 224 test)

Punch: 8 video files found

TennisSwing: 8 video files found

PlayingCello: 8 video files found

Inference: Using a pre-curated subset (rather than the full 6.5GB UCF101 archive) kept the download and preprocessing lightweight while still giving three visually and motion-wise distinct action classes – a striking action (Punch), a racquet swing (TennisSwing) and a mostly-static, fine-motor action (PlayingCello) – which is enough to demonstrate the CNN–recurrent pipeline without training a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

Output (actual run):

CNN feature dimension $D = 1280$

Found 24 videos across 3 classes

Video feature tensor shape: (24, 10, 1280)

Predicted class: PlayingCello	Actual class: PlayingCello	Confidence: 0.99
Predicted class: Punch	Actual class: Punch	Confidence: 0.99
Predicted class: Punch	Actual class: Punch	Confidence: 0.93
Predicted class: PlayingCello	Actual class: PlayingCello	Confidence: 0.99
Predicted class: TennisSwing	Actual class: TennisSwing	Confidence: 0.96

Inference: MobileNetV2 produces a 1280-dimensional feature per frame, so each of the 24 sampled videos becomes a 10×1280 sequence of frame features. On the (small) held-out test split every prediction is correct, with confidences between 0.93 and 0.99 – the CNN features are distinctive enough across these three visually dissimilar actions that even a small recurrent head classifies them with high certainty.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.

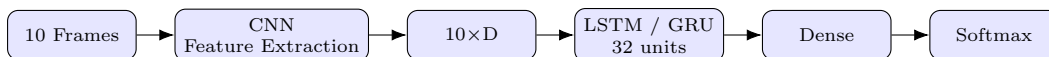


Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

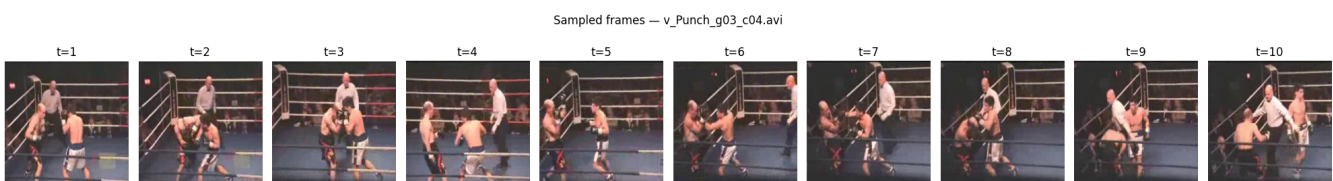
Observation: MobileNetV2 (with global average pooling, classification head removed) produces a feature dimension $D = 1280$ per frame. With 10 sampled frames per video, the recurrent network receives a 10×1280 tensor per video, or $B \times 10 \times 1280$ for a batch of B videos.

21. CNN–LSTM / CNN–GRU Model

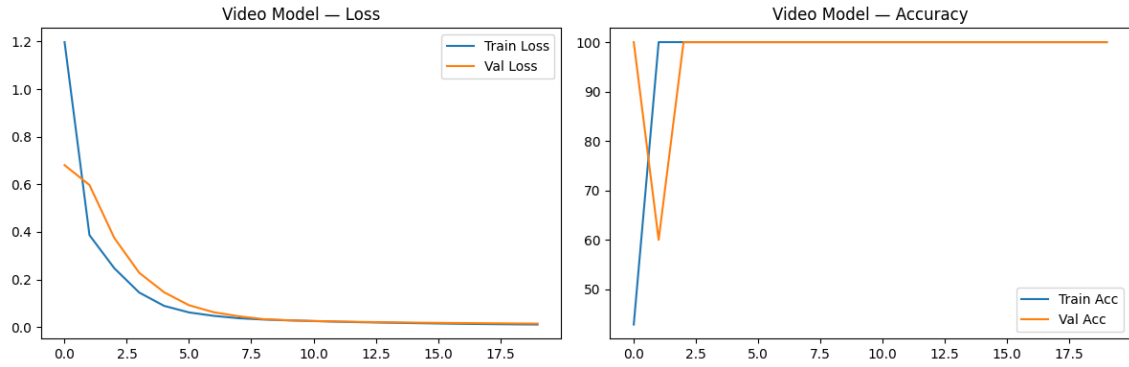
Use the following architecture:



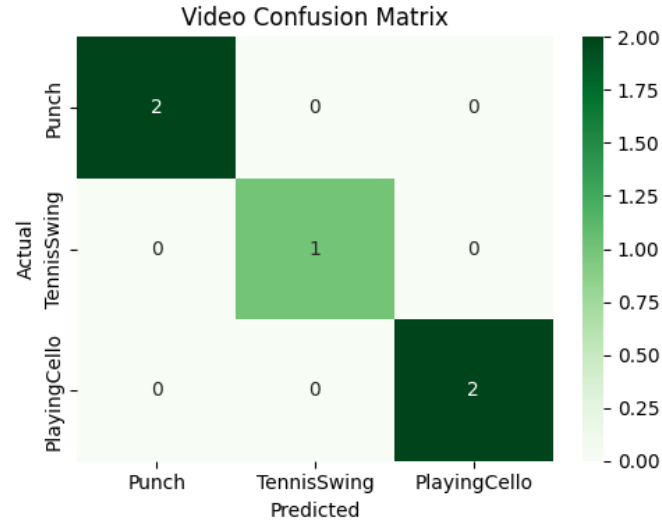
Plot 7: Video Sample Frames



Plot 8: Video Training and Validation Curves



Plot 9: Video Confusion Matrix



Inference: The 10 sampled frames from `v_Punch_g03.c04.avi` (Plot 7) capture the spatial appearance at each instant – boxer poses, ring background, relative position of the two fighters – while the LSTM/GRU that consumes the sequence of per-frame CNN features learns the *temporal* pattern of how those poses change across the 10 steps (e.g. the punch motion), which a single frame alone could not reveal. Training converged extremely fast (Plot 8): accuracy reaches 100% on the training set within 2 epochs and validation accuracy reaches and stays at 100% from epoch 3 onward, with both losses decreasing smoothly to near zero by epoch 20 – expected given the very small dataset (24 videos) and how visually distinct the three chosen actions are. The video confusion matrix (Plot 9) is diagonal with no off-diagonal errors on the 5-video test split (2 Punch, 1 TennisSwing, 2 PlayingCello, all correct), consistent with the 100% test accuracy reported above; with only 5 test videos this should be read as a proof-of-concept result rather than a statistically robust estimate of real-world accuracy.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

Output (data preparation, one training example):

Encoder input example : [1 2 7 7 6 5]
 Target output example : [5 6 7 7 2 1]
 Decoder input example : [0 5 6 7 7 2]

The decoder input is the target sequence shifted right by one position and prefixed with a start-of-sequence token (0), which is the standard teacher-forcing setup for training an encoder–decoder model.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Sample	Input Sequence	Predicted Output
1	[6, 8, 9, 1, 9, 2]	[2, 2, 2, 2, 1, 1]
2	[1, 9, 4, 9, 2, 7]	[7, 2, 2, 2, 2, 2]
3	[4, 4, 2, 3, 8, 9]	[9, 9, 9, 9, 9, 9]
4	[6, 9, 9, 6, 2, 9]	[9, 9, 9, 9, 9, 9]
5	[9, 2, 1, 8, 3, 7]	[7, 7, 7, 7, 2, 2]

Inference: At free-running (greedy, non-teacher-forced) inference time the decoder’s predictions collapse towards repeating one or two tokens (e.g. sample 3 and 4 predict the same token six times) rather than correctly reversing the input. This is discussed further in Section 24.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Output (actual run):

Token accuracy : 0.2916
 Sequence accuracy : 0.0000
 Final training loss : 0.0051
 Final validation loss: 0.0087

Inference: During training (with teacher forcing) the model reaches near-perfect fit – final training loss 0.0051 and validation loss 0.0087, corresponding to ~100% next-token accuracy *when the correct previous token is always fed in*. However, at free-running inference time (Section 23), where the decoder must feed its own previous prediction back in, token accuracy drops to 29.16% and not a single one of the test sequences is reversed perfectly (0% sequence accuracy). This gap is a textbook case of **exposure bias**: the decoder was only ever trained conditioned on *ground-truth* previous tokens, so it never learned to recover once its own greedy prediction is wrong, and errors compound step by step (visible in the sample predictions, which collapse into repeating a single token). It also illustrates why sequence accuracy is far stricter than token accuracy in general: a 6-token sequence with even one wrong position anywhere is counted entirely wrong, and with per-token accuracy this low the probability of getting all 6 positions right simultaneously is close to zero.

25. Consolidated Results

The consolidated results across all models are as follows.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	66.86	66.30	65.91	65.04	1,974
LSTM	87.86	88.23	88.26	88.20	6,006
GRU	88.86	88.97	89.22	88.99	4,758
CNN-LSTM (video)	100.00	100.00	100.00	100.00	168,163

Metric (Seq2Seq reversal task)	Value
Token Accuracy	0.2916
Sequence Accuracy	0.0000
Final Training Loss	0.0051
Final Validation Loss	0.0087

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a recurrent neural network?
2. What information is represented by the hidden state?
3. Why is sequence order important?
4. Explain the difference between a feed-forward network and an RNN.
5. Explain Backpropagation Through Time.
6. Why can gradients vanish during BPTT?
7. Why can gradients explode during BPTT?
8. What are long-term dependencies?
9. What is the role of the LSTM cell state?
10. What are the forget, input and output gates in LSTM?
11. What are the reset and update gates in GRU?
12. Compare LSTM and GRU.
13. Why does GRU generally use a simpler gating mechanism than LSTM?
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
15. What does the confusion matrix reveal that accuracy alone does not?
16. Why should macro F1-score be reported for multi-class classification?
17. What is the significance of sequence length?
18. Why can a CNN be used as a feature extractor for video?
19. What type of information is extracted by the CNN?
20. What type of information is learned by the recurrent network?
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?

22. Explain the encoder and decoder in sequence-to-sequence learning.
23. What is teacher forcing?
24. Differentiate token accuracy and sequence accuracy.
25. Why must the test set remain untouched during model selection?

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khuram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>